Streams en Java

Esta obra, cuyo autor es Marcos Martínez Rus, ha sido publicada bajo una
licencia de Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional

ÍNDICE

1. Introducción....3

2. Clase Persona....4

3. Métodos....6

3.1 allMatch()....6

3.2 anyMatch()....7

3.3 filter()....7

3.3.1 count()....8

3.4 distinct()....8

3.5 max() y min()....9

3.6 skip()....9

3.7 limit()....10

3.8 takeWhile....10

3.9 .findFirst()....10

3.10 sorted()....11

3.11 map()....12

3.12 Métodos matemáticos....12

4. Referencia a métodos o métodos referenciados....13

1. Introducción

Los `Streams` son una herramienta que nos permite trabajar de forma muy eficiente, legible y cómoda con flujos de datos. En nuestro vocabulario, nos permite trabajar con colecciones y diccionarios de forma muy eficiente y con código conciso.

Para ello, únicamente debemos concatenar `.stream()` a la colección o diccionarios con el que estemos trabajando y, apoyándonos en expresiones lambda y métodos propios de los flujos de datos, podremos conseguir grandes resultados con poco esfuerzo.

Supongamos que tenemos una lista de enteros y queremos quedarnos únicamente con todos aquellos que sean superiores a 10. Podemos implementar el siguiente código para ello:

```
List<Integer> numeros = List.of(5, 8, 10, 11, 9, 26);
List<Integer> mayoresDe10 = numeros.stream().filter(n -> n > 10).toList();
```

Donde *mayoresDe10* únicamente contendrá los números 11 y 26.

Si nos fijamos, el método `filter()` nos permite filtrar una colección de datos haciendo uso de un predicado. Podríamos haberlo implementado también así:

```
Predicate<Integer> filtroMayoresDe10 = n -> n > 10;
List<Integer> mayoresDe10 = numeros.stream().filter(filtroMayoresDe10).toList();
```

Muchas de las funcionalidades que se utilizan con los Streams y que están incluidas en la librería `java.util.stream` son interfaces funcionales y permiten por lo tanto el uso de expresiones lambda.

2. Clase Persona

Para las explicaciones de este tema, vamos a utilizar la siguiente clase que modela los datos de una persona.

```
public class Persona {
	private int dni;
	private String nombre;
	private String apellidos;
	private LocalDate fechaNacimiento;

	public Persona(int dni, String nombre, String apellidos, LocalDate fechaNacimiento)
	{
		super();
		this.dni = dni;
		this.nombre = nombre;
		this.apellidos = apellidos;
		this.fechaNacimiento = fechaNacimiento;
	}

	public int getDni() {
		return dni;
	}

	public void setDni(int dni) {
		this.dni = dni;
	}

	public String getNombre() {
		return nombre;
	}

	public void setNombre(String nombre) {
		this.nombre = nombre;
	}

	public String getApellidos() {
		return apellidos;
	}
```

```java
	public void setApellidos(String apellidos) {
		this.apellidos = apellidos;
	}

	public LocalDate getFechaNacimiento() {
		return fechaNacimiento;
	}

	public void setFechaNacimiento(LocalDate fechaNac) {
		this.fechaNacimiento = fechaNac;
	}

	public int edad() {
		return (int) ChronoUnit.YEARS.between(getFechaNacimiento(), LocalDate.now());
	}

	@Override
	public String toString() {
		return "Persona [dni=" + dni + ", nombre=" + nombre + ", apellidos=" +
				apellidos + ", fechaNacimiento=" + fechaNacimiento + "]";
	}

	@Override
	public boolean equals(Object obj) {
		if (this == obj) return true;
		if (obj == null) return false;
		if (getClass() != obj.getClass()) return false;

		Persona other = (Persona) obj;
		return Objects.equals(apellidos, other.apellidos) && dni == other.dni
				&& Objects.equals(fechaNacimiento, other.fechaNacimiento)
				&& Objects.equals(nombre, other.nombre);
	}

	@Override
	public int hashCode() {
		return Objects.hash(apellidos, dni, fechaNacimiento, nombre);
	}
}
```


Profesor: Marcos Martinez Rus

Para el desarrollo del tema y la realización de los ejemplos vamos a suponer que tenemos la siguiente lista de personas.

```
List<Persona> personas = new ArrayList<Persona>();
personas.add(new Persona(11, "Paco", "Gento", LocalDate.of(1933, 10, 21)));
personas.add(new Persona(9, "Alfredo", "Di Stéfano", LocalDate.of(1926, 7, 4)));
personas.add(new Persona(5, "Zinedine", "Zidane", LocalDate.of(1972, 6, 23)));
personas.add(new Persona(10, "Luka", "Modric", LocalDate.of(1985, 9, 9)));
personas.add(new Persona(7, "Raúl", "González Blanco", LocalDate.of(1977, 6, 27)));
```

3. Métodos

A continuación, vamos a ver una serie de métodos que nos van a ser muy útiles para el manejo de información. Para ello, recuerda, debemos obtener primero un `stream`.

3.1 allMatch()

Este método nos permite verificar si todos los elementos de nuestra colección, cumplen una determinado condición haciendo uso de un predicado (`Predicate`).

Si queremos saber si todas las personas tienen 18 años o más, podemos implementar el siguiente código:

```
boolean todosSonMayorDeEdad = personas.stream().allMatch(p -> p.edad() >= 18);
```

Si deseamos saber si todas las personas tienen apellidos de más de 8 caracteres, podemos implementar lo siguiente:

```
boolean todosTienenApellidosDeMasDe8Caracteres =
                                personas.stream().allMatch(p -> p.getApellidos().length() > 8);
```

3.2 anyMatch()

Este método nos permite verificar si algún elemento del flujo de datos, cumple una determinada condición implementada mediante un predicado. Puede haber más de uno que lo cumpla, pero con que haya uno solo, ya devuelve verdadero.

Si queremos saber si hay alguna persona mayor de edad, podemos implementar el siguiente código:

```
boolean algunoMayorDeEdad = personas.stream().anyMatch(p -> p.edad() >= 18);
```

Si deseamos saber si alguna de las personas tiene apellidos de más de 8 caracteres, podemos implementar lo siguiente:

```
boolean algunoTienenApellidosDeMasDe8Caracteres =
                                personas.stream().anyMatch(p -> p.getApellidos().length() > 8);
```

3.3 filter()

Este método nos permite filtrar nuestros datos, es decir, quedarnos únicamente con los elementos que cumplen una determinada condición implementada mediante un predicado. **Este método es muy útil para el manejo de flujos de datos (información)**.

Si queremos obtener las personas que tienen más de 85 años, podemos implementar el siguiente código:

```
List<Persona> mayoresDe85 = personas.stream().filter(p -> p.edad() > 85).toList();
```

Fíjate que necesitamos realizar un toList() al final de nuestra expresión de código. Ello es debido a que, en este caso, devolvemos más de un dato: puede haber más de una Persona que cumpla la condición.

3.3.1 count()

Este método nos permite saber el número de elementos que contiene el flujo de datos. Supongamos que queremos conocer cuántas personas han nacido en 1977. Podríamos implementar el siguiente código el cual es eficaz (hace lo que tiene que hacer):

```
List<Persona> nacidosEn1977 =
        personas.stream().filter(p -> p.getFechaNacimiento().getYear() == 1977).toList();
int numNacidosEn1977 = nacidosEn1977.size();
```

Lo anterior implica que, primero, tenemos que generar una lista de personas con todos sus atributos cargándola en memoria y, posteriormente, comprobar el número de elementos que tiene la lista (`.size()`). En realidad, el nombre, los apellidos y demás atributos no nos importan para esta funcionalidad, lo único que deseamos es saber cuántos cumplen una determinada condición.

Una forma eficaz y más eficiente es aprovecharnos del procesamiento interno que hacen los Streams antes de materializar la instanciación de objetos, es decir, le indicamos al `stream()` que únicamente necesitamos saber cuántos elementos cumplen la condición y que no es necesario generar los objetos. Para ello utilizamos el método `count()` de esta forma:

```
long numNacidosEn1977 = personas.stream().filter(p -> p.getFechaNacimiento().getYear() == 1977).count();
```

3.4 distinct()

Este método nos devuelve los elementos que son diferentes. Por ejemplo, si tenemos varias veces la misma Persona en la lista, únicamente aparecerá una vez en la lista que nos devuelve. ¿Y cómo sabe si son iguales o no? ¡A través de los métodos `equals()` y `hashCode()` que tan bien sabemos manejar ya!. Un ejemplo de código sería el siguiente:

```
List<Persona> unicas = personas.stream().distinct().toList();
```

3.5 max() y min()

Nos permiten, haciendo uso de un comparador (`Comparator`), obtener el valor máximo y mínimo, respectivamente, según el criterio que deseemos.

Por ejemplo, si deseamos obtener de entre todas las personas, la de menos edad y la de más edad, podemos implementar el siguiente código:

```
Comparator<Persona> comparadorFechaNacimiento = (p1, p2) ->
                                p1.getFechaNacimiento().compareTo(p2.getFechaNacimiento());

Persona menor = personas.stream().max(comparadorFechaNacimiento).get();
Persona mayor = personas.stream().min(comparadorFechaNacimiento).get();
```

En este caso, debemos hacer uso del método `get()` ya que lo que se devuelve es un `Optional<Persona>` no una `Persona`. Puedes leer más sobre `Optional` de Java aquí.

3.6 skip()

Este método es una operación intermedia que nos permite descartar los n primeros elementos del flujo de datos. Es importante resaltar que n, como es lógico, debe ser un número positivo y si es superior al número de elementos del flujo de datos, lo que se devolverá es una secuencia vacía (*empty*).

Por ejemplo, si queremos obtener un listado de personas descartando a las tres primeras, podemos implementar el siguiente código:

```
List<Persona> resto = personas.stream().skip(3).toList();
```

CC BY NC SA

3.7 limit()

Este método es una operación intermedia que nos permite obtener un flujo de datos que no supere el tamaño n de elementos.

Por ejemplo, si queremos obtener un listado de las tres primeras personas descartando al resto, podemos implementar el siguiente código:

```
List<Persona> podio = personas.stream().limit(3).toList();
```

3.8 takeWhile

Este método nos permite ir quedarnos con todos los elementos del flujo de datos hasta que se cumple una determinada condición. Supongamos que queremos ir recorriendo nuestra lista (y quedándonos con los objetos de la misma) hasta que alguna persona haya nacido en 1985 o después. Lo podemos hacer de la siguiente forma:

```
List<Persona> seleccion =
       personas.stream().takeWhile(p -> p.getFechaNacimiento().getYear() < 1985).toList();
```

3.9 .findFirst()

Este método nos permite obtener el primer elemento del flujo de datos. **Este método es muy utilizado para obtener el primer elemento que cumple la condición de la regla de negocio deseada**.

Si queremos obtener la primera persona de nuestro flujo de datos, podríamos hacer lo siguiente:

```
Persona primero = personas.stream().findFirst().get();
```

A lo anterior, en principio, no le encontramos ninguna ventaja. Sigamos leyendo...

Llegados a este punto, considero importante introducir un concepto muy útil: los métodos de los Streams están diseñados para ser concatenados entre sí lo que deriva en conseguir operaciones complejas con código muy intuitivo.

Por ejemplo, supongamos que queremos obtener la primera Persona que tiene un número de DNI impar. Recuerda que el orden en la lista, por defecto, es el orden de inserción. Para ello, primero deberemos filtrar por número impar de DNI y, posteriormente, obtener el primer elemento. Lo podríamos hacer así:

```
Persona primeroConDniImpar =
                    personas.stream().filter(p -> p.getDni() % 2 != 0).findFirst().get();
```

3.10 sorted()

Este método nos permite ordenar un flujo de datos por su orden natural (el indicado en el método `compareTo()` al implementar la interfaz `Comparable` en la clase). Si la clase no es `Comparable`, se produce una excepción. También podemos indicarle, mediante un `Comparator`, el criterio de comparación.

Podríamos implementar el siguiente código para mostrar por consola a las persona según su orden natural:

```
            personas.stream().sorted().forEach(p -> System.out.println(p));
```

Sin embargo, a este método se le saca mucha más utilidad cuando lo combinamos con otros métodos del stream(). Por ejemplo, siguiendo con el ejemplo del método anterior, supongamos que queremos obtener la Persona que tiene el número de DNI impar más pequeño. Lo podríamos hacer así:

```
Persona personaDniMasBajo = personas.stream()
                            .filter(p -> p.getDni() % 2 != 0)
                            .sorted((p1,p2) -> Integer.compare(p1.getDni(), p2.getDni()))
                            .findFirst()
                            .get();
```

3.11 map()

Este método es uno de los más útiles al trabajar con `Streams`. Nos permite realizar una transformación rápida de los datos y muy directa sobre el flujo original, es decir, transformar el elemento original a otro tipo de elemento.

Supongamos que queremos quedarnos únicamente con las edades de las personas. Podemos implementar lo siguiente:

```
List<Integer> edades = personas.stream().map(p -> p.edad()).toList();
```

3.12 Métodos matemáticos

Se puede dar el caso que nos interese calcular la suma de las edades (`sum()`), la media (`average()`), la edad máxima (`max()`) o mínima (`min()`), por ejemplo. Para ello podemos hacer uso de los métodos `mapToInt()` o `mapToDouble()` de los Streams y, posteriormente, aplicar la función matemática deseada.

Para verlo más claro, vamos a ver cómo podríamos obtener la suma de las edades de todas las personas:

```
int sumaEdades = personas.stream().mapToInt(p -> p.edad()).sum();
```

Importante. Hay otros muchos métodos, algunos de ellos muy potentes así que es recomendable echar un ojo a la documentación de Java. No obstante, con los que acabamos de ver, ya podemos obtener funcionalidades complejas con un código muy legible.

4. Referencia a métodos o métodos referenciados

Con la inclusión de las expresiones lambda podemos hacer uso de una característica conocido como referencia a métodos o métodos referenciados, la cual nos permite referirnos a un método sin llamar explícitamente a su ejecución.

Existen diferentes tipos. Nosotros nos vamos a centrar en trabajar con la sintaxis siguiente:

```
NombreClase::nombreMetodo
```

De esta manera, podemos ver un par de ejemplos de uso. Ambas notaciones obtienen el mismo resultado:

Si queremos mostrar todas las personas por pantalla, podemos implementar el siguiente código:

```
personas.forEach(persona -> System.out.println(persona));
```

O implementarlo mediante su referencia a método:

```
personas.forEach(System.out::println);
```

Por otro lado, también podemos sustituir la siguiente sentencia que obtiene la suma de las edades de todas las personas:

```
int sumaEdades = personas.stream().mapToInt(p -> p.edad()).sum();
```

Por la siguiente sentencia equivalente:

```
int sumaEdades = personas.stream().mapToInt(Persona::edad).sum();
```

CC BY NC SA